

CodeSheriff: Design and Development of an Agentic Security Auditor for an SFI-Enabled Confidential CI/CD Execution Layer Using Bayesian Vulnerability Inference

Shubhankar Vishnu Patil

Dept. of Information Technology

G H Raisoni College of Engineering and Management
Pune, India

shubhankarp258@gmail.com

Vivek Dnyaneshwar Joshi

Dept. of Information Technology

G H Raisoni College of Engineering and Management
Pune, India

vivekdjoshi05@gmail.com

Yash Suresh Kodgirwar

Dept. of Information Technology

G H Raisoni College of Engineering and Management
Pune, India

yashkodgirwar@gmail.com

Nirmalya Mukhopadhyay

Dept. of Information Technology

G H Raisoni College of Engineering and Management
Pune, India

Kunal Kundu

Dept. of Information Technology

G H Raisoni College of Engineering and Management
Pune, India

Abstract—Modern software development increasingly relies on Continuous Integration/Continuous Deployment (CI/CD) pipelines to deliver software rapidly, but automated security review at the pull request stage remains limited. Existing security tools rely on rigid static analysis or homogeneous multi-agent approaches, which may generate false positives or overlook complex vulnerabilities. This project will present CodeSheriff, an agentic security auditor that will analyze GitHub pull requests using structural, semantic, contextual, and SFI-based confidential execution analysis. A Bayesian inference engine will fuse evidence from multiple agents to generate confidence-aware vulnerability assessments, while an automated patch generation and verification module will recommend and validate fixes through GitHub and a web dashboard. The system will enhance CI/CD security by delivering reliable, explainable, and statistically grounded vulnerability detection while reducing manual review effort.

Index Terms—vulnerability detection, multi-agent systems, software security, continuous integration and continuous deployment,

Integration/Continuous Deployment (CI/CD) that's builds and runs the code.

Solutions that exist today don't fully solve this problem. Traditional static analysis approaches identify fixed patterns but are unable to catch tricky ones, so they generate a lot of false alarms. Some recent tools use multiple artificial intelligence (AI) agents, but all these agents basically read the same code in the same way, so they don't really give different opinions. When these approaches combine results, they just use simple voting, which doesn't tell you how confident the final answer actually is.

In this paper, we are proposing CodeSheriff to fix this problem. It uses four different types of agents that look at the code from different perspectives. Static agent checks the code structure, the meaning of the code is understood by the semantic agent, the context agent checks whether accepting this PR could bring back any vulnerability from previously accepted PR, and the runtime agent actually watches the code run inside a safe, isolated environment using software fault isolation (SFI). All this information is then combined using a Bayesian model, which gives a clear probability of whether the code is vulnerable, along with how confident the system is. This same safe environment also protects the CI/CD workflow, since it only releases secret keys when it is sure the correct, verified code is running.

The rest of this paper is structured as follows. Section II talks about related work already done in this area. Section III explains our proposed method and how the system is designed.

I. INTRODUCTION

In today's software industry, developers frequently push code to the main branch through Pull Requests (PRs). However, security analysis of each Pull Request is still largely performed manually, making the review process more time-consuming. Teams either use basic approaches that only catch known, simple mistakes, or they don't have enough time to properly review every pull request. This is a big problem because even one missed vulnerability can put the whole application at risk once the code is merged and deployed. On top of this, attackers are now also targeting the Continuous

II. LITERATURE SURVEY

Early learning-based studies explored structural and sequential representations for vulnerability detection. Li and Yang [1] compared sequence and graph-based deep learning methods, identifying challenges in capturing long-range dependencies and generalizing to noisy data. CODE-SMASH [2] and XTransformer [3] employed hierarchical neural and transformer architectures to capture multi-level code representations and improve similarity-based detection, while remaining sensitive to computational cost and dataset characteristics. Abbasi and Shameli-Sendi [4] used Code Property Graph metrics with machine learning for interpretable vulnerability and vulnerability-type prediction, but their approach was limited to C/C++. DeepSecure [5] combined CodeBERT with bidirectional long short-term memory (BiLSTM) and convolutional neural network (CNN) layers for python vulnerability detection, incorporating robustness and confidence calibration, but remained language-specific and affected by noisy labels.

Transformer and large language model (LLM) based approaches have subsequently improved semantic analysis of source code. Shestov *et al.* [6] fine-tuned WizardCoder using parameter-efficient learning for vulnerability classification, but remained limited to function-level analysis. Tamberg and Bahsi and Gnieciak and Szandala [7], [8] benchmarked LLMs against vulnerability detection tasks and traditional static analysis tools, showing strong potential but persistent false positives, hallucinations, localization errors, and inference costs. Gülmez [9] surveyed code-generation LLMs and highlighted challenges in structural correctness, repository-level reasoning, and security. Chen *et al.* [10] and Su *et al.* [11] similarly identified limitations involving context windows, hallucination, dataset quality, generalization, and interpretability. Germano *et al.* [12] reviewed LLM-based vulnerability detection, repair, and explanation, noting that research remains more focused on detection than automated repair and explanation.

To address the limitations of isolated code analysis, recent studies have incorporated contextual and repository-level information. Antal *et al.* [13] evaluated retrieval augmented generation (RAG) for real-world Java vulnerability detection using retrieved vulnerable and secure examples, but observed false positives and retrieval sparsity. AegisGuard [14] incorporated contextual information for semantic vulnerability detection and risk stratification, while SCOUT [15] enabled interactive repository exploration to retrieve relevant code context. Göçmen *et al.* [16] integrated repository history and dependency analysis into pull-request reviews for change-impact estimation, although static dependency analysis cannot fully capture dynamic behavior.

Multi-agent approaches have further introduced collaborative security reasoning. Wei *et al.* [17] used specialized roles and collaborative reasoning for vulnerability auditing, while MAVUL [18] combined vulnerability analysis, security architecture review, and iterative evaluation. Reddy *et al.* [19] explored knowledge-based agentic AI for automated vulnerability triage, and Syed *et al.* [20] investigated an observe–reason–

act–verify framework for autonomous software-supply-chain defense. These studies demonstrate the potential of agentic security workflows, while also highlighting challenges such as reduced performance on complex cases, token overhead, and the need for human intervention.

However, existing approaches do not effectively combine structural, semantic and contextual analysis within a unified vulnerability detection framework. We integrate these approaches through specialized agents and confidence-aware vulnerability inference, while providing explanations and patch recommendations within the GitHub pull-request workflow.

III. PROPOSED METHODOLOGY

A vulnerability can hide in the structure of the code, in its intent, in a departure from the repository’s own habits, or only in what the code does when it runs. CodeSheriff therefore analyses every changed function with four independent agents – Structural, Semantic, Context and Runtime – that never see each other’s output, and combines their evidence using Bayesian inference instead of voting. Each agent contributes one likelihood ratio that updates a shared posterior probability, so the reported confidence is calibrated rather than asserted.

A. Mathematical Model

The framework evaluates a pull request in five stages: change extraction, independent agent analysis, evidence categorisation, Bayesian fusion and the alert decision. The unit of analysis is one changed function u , obtained by mapping every line that differs between the base and head versions of a file to the outermost function containing it.

1) *Structural Agent*: The Structural Agent follows data through a definition–use graph from each function parameter, which is treated as untrusted, to a security-sensitive operation (a *sink*) such as `cursor.execute` or `os.system`, and checks whether the path is sanitized. When an unsanitized path reaches a sink, the structural score is

$$S_{\text{str}} = \text{clamp}_{[0,1]}(0.45 + \delta + 0.15\nu - 0.25\varsigma - 0.20\tau - \lambda) \quad (1)$$

where δ is 0.20 for a critical sink and 0.10 for a high-severity sink, $\nu = 1$ for a network-facing unit, $\varsigma = 1$ when a partial sanitizer lies on the path, $\tau = 1$ in a test file, and $\lambda \leq 0.05$ is a small penalty for long paths. Semgrep runs as a second backend on the same unit, and the larger of the two scores is kept, because both read the same source text with the same technique.

2) *Semantic Agent*: The Semantic Agent uses a large language model to infer the functional intent of the code, its trust boundaries and the security invariant that the change breaks. The model is sampled $n = 3$ times with different seeds, and a sample is accepted only if the weakness class is in scope, the file matches, the quoted sink appears verbatim in the source and the reported lines lie inside the unit. For weakness class c , the semantic score is the agreement across accepted samples:

$$S_{\text{sem}}(c) = \frac{m_c}{n} \quad (2)$$

where m_c is the number of accepted samples reporting c and n is the total number of samples. A single sample cannot separate confidence from chance, which is why agreement, and not the model's own wording, is used as the score.

3) *Context Agent*: The Context Agent retrieves the repository's own merged history using local sentence embeddings stored in pgvector and reports a security control γ that the history applies and the changed function does not. The contextual score is

$$S_{\text{ctx}}(\gamma) = b \cdot \min\left(1, \frac{\bar{s}}{0.85}\right) \quad (3)$$

where $\bar{s}$ is the mean similarity of the retrieved supporting functions, and $b = 0.85$ when the same function established γ in an earlier merge or three or more sibling functions apply it, and $b = 0.65$ when exactly two siblings apply it. A single neighbour is treated as coincidence and reports nothing.

4) *Runtime Agent*: The Runtime Agent executes the changed function once inside a WebAssembly sandbox built on Wasmtime and WASI, which enforces software fault isolation at the instruction level: the network is denied at link time, the environment is empty and fuel, time and memory are capped. Each parameter is bound to a unique random token, so a value is untrusted exactly when one of those tokens appears inside it. If such a value reaches a sink, the agent reports a detection with the fixed score $S_{\text{rt}} = 0.85$; a constant is used because this is a direct observation and not an estimate. If the value reached the sink only because the sandbox had to answer a guard it could not evaluate, the agent abstains.

5) *Bayesian Fusion and Decision*: Every agent returns a *detection* with a score, a *silence* (it ran and found nothing) or an *abstention* (it could not run). Raw scores from different agents are not directly comparable, so each statement is first mapped to one shared evidence cell:

$$z_a = \begin{cases} \text{HIGH}, & S_a \geq 0.8 \\ \text{MEDIUM}, & 0.5 \leq S_a < 0.8 \\ \text{LOW}, & S_a < 0.5 \\ \text{SILENCE}, & \text{agent ran, found nothing} \end{cases} \quad (4)$$

Each cell is weighted by how reliably it has predicted vulnerability in labelled data, through a likelihood ratio estimated with Laplace smoothing:

$$\Lambda_a(z) = \frac{P(z | V)}{P(z | \neg V)} \approx \frac{(k_V + 1)/(N_V + 4)}{(k_S + 1)/(N_S + 4)} \quad (5)$$

where k_V and k_S are the numbers of vulnerable and safe examples that fell into cell z , and N_V and N_S are the corresponding totals. An abstention, and any cell never observed, takes $\Lambda_a = 1$, so an agent that could not look leaves the result unchanged instead of being read as a vote of "safe". Ratios are clamped to $[0.05, 20]$ so that no single agent can dominate. The prior odds are then updated multiplicatively and converted back into a probability:

$$P(V | E) = \frac{O}{1 + O}, \quad O = \frac{\pi}{1 - \pi} \prod_{a \in \{S, M, C, R\}} \Lambda_a(z_a) \quad (6)$$

where π is the prior probability of a vulnerability and O is the posterior odds. The product always contains four factors, one per agent, and a silence contributes a factor below 1, so the probability can fall as well as rise. The finding is finally labelled

$$\text{Verdict} = \begin{cases} \text{ALERT}, & P(V | E) \geq \tau \\ \text{NO ALERT}, & \text{otherwise} \end{cases} \quad (7)$$

where τ is the alert threshold fitted on labelled data. With $\pi = 0.03$ and $\tau = 0.227$, a single strong semantic detection that no other agent contradicts is exactly sufficient to raise an alert, while a medium structural, a high semantic and a high runtime detection together give $P(V | E) \approx 0.97$.

B. Algorithms

The framework is implemented as two algorithmic stages: independent evidence generation by the four agents, and Bayesian fusion with the alert decision and patch handoff.

Algorithm 1: Agent Evidence Generation

Input: Changed function u
Output: Evidence $E = \{e_S, e_M, e_C, e_R\}$

```

1 foreach agent  $a \in \{S, M, C, R\}$  do
2   if  $a$  cannot analyse  $u$  then
3      $e_a \leftarrow \text{ABSTENTION}(\text{reason});$ 
4   else
5      $F_a \leftarrow$  findings of  $a$  on  $u$ ; // Eqs. (1) -- (3)
6     if  $F_a \neq \emptyset$  then  $e_a \leftarrow \text{DETECTION}(S_a, c);$ 
7     else  $e_a \leftarrow \text{SILENCE};$ 
8 return  $E$ ;
```

Algorithm 1 runs the four agents independently on one changed function. Every agent always returns a statement, so a failure is recorded as an abstention and is never confused with "analysed and found nothing".

Algorithm 2: Bayesian Fusion, Decision and Patching

Input: Evidence E , ratios Λ , prior π , threshold τ
Output: $P(V | E)$, Verdict Y , patch F

```

1  $O \leftarrow \pi / (1 - \pi);$ 
2 foreach agent  $a$  do
3    $z_a \leftarrow$  cell of  $e_a$ ; // Eq. (4)
4    $O \leftarrow O \times \Lambda_a(z_a);$  //  $\Lambda_a = 1$  if abstained
5  $P(V | E) \leftarrow O / (1 + O);$ 
6  $Y \leftarrow \text{ALERT}$  if  $P(V | E) \geq \tau$  else  $\text{NO ALERT};$ 
7 if  $Y = \text{ALERT}$  then
8    $F \leftarrow \text{DRAFTPATCH}(u, E)$ , verified by the deterministic agents;
9   publish  $F$  as a suggestion only if it lies inside the changed lines;
10 else  $F \leftarrow \emptyset;$ 
11 return  $P(V | E)$ ,  $Y$ ,  $F$ ;
```

Algorithm 2 multiplies the prior odds by one fitted ratio per agent, converts the result into a probability and compares it with the threshold. An alerted finding is passed to the patch stage, where a drafted repair must parse, keep the function signature, use only names the file defines, change the code,

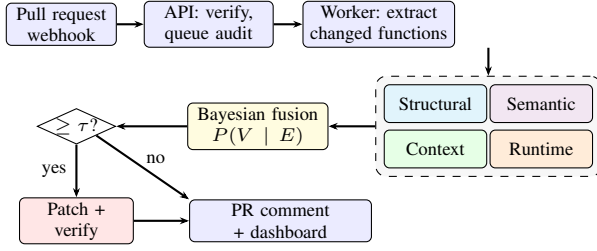


Fig. 1. Workflow of CodeSheriff. The four agents run blind.

and survive re-examination by the Structural and Runtime agents before it is suggested. CodeSheriff only suggests; it never commits or merges.

C. Technical Workflow

As illustrated in Fig. 1, the CodeSheriff workflow consists of five stages:

- 1) **Trigger:** opening or updating a pull request sends a GitHub webhook. The API verifies its HMAC-SHA256 signature, records an audit, places it on a queue and replies within three seconds.
- 2) **Change Extraction:** a worker downloads the base and head versions of each changed file and builds one unit per changed function.
- 3) **Agent Analysis:** the four agents analyse every unit independently and produce structured evidence (Algorithm 1).
- 4) **Fusion and Decision:** the inference engine computes $P(V | E)$ and the verdict, and alerted findings are passed to the patch stage (Algorithm 2).
- 5) **Reporting:** one comment summarises the audit on the pull request, and the verdict, probability, per-agent evidence and patch outcome are stored for the dashboard.

D. Implementation

CodeSheriff is a Python 3.12 monorepo. The API and the worker run as separate processes with separate credentials: the API holds the webhook secret, the worker holds the model key, and a queued task carries only an audit identifier. Source code is held in memory only; the database stores findings, evidence and hashes. The main components are:

- **Backend:** FastAPI with Celery and Redis for queued audits.
- **Database:** PostgreSQL 16 with pgvector, SQLAlchemy, Alembic.
- **Agents:** tree-sitter and a networkx taint engine with Semgrep; a hosted LLM at $n = 3$ samples; bge-small-en-v1.5 embeddings; Wasmtime with a WASI build of CPython.
- **Dashboard:** Next.js with TypeScript and Recharts, read-only.
- **Quality:** 1249 tests, strict type checking, and import rules that stop any agent from importing another agent or the database.

TABLE I
DETECTION AND CALIBRATION ON THE VALIDATION SPLIT. BEST PER COLUMN IN BOLD.

System	ECE ↓	Brier ↓	P ↑	R ↑	F_1 ↑
Semgrep [21]	—	—	—	—	—
Bandit [22]	—	—	—	—	—
Single LLM [8]	—	—	—	—	—
Prior only	0.000	0.029	0.000	0.000	0.000
Majority vote [18]	0.141	—	0.076	1.000	0.142
CodeSheriff (full)	0.027	0.013	1.000	0.857	0.923

IV. EXPERIMENTAL EVALUATION

A. Environmental Setup

We evaluate the prototype on a system with 16 GB RAM and an Intel Core i5 processor. The implementation uses Python 3.12, Semgrep, Tree-sitter, Sentence Transformers, pgvector, and a Wasmtime/WASI sandbox. The system was evaluated based on calibration (expected calibration error, Brier score) and detection accuracy (precision, recall, F_1).

B. Comparative Analysis

This subsection evaluates CodeSheriff’s detection performance and calibration quality against two classes of comparators: naive inference baselines and external detectors evaluated on the same corpus, and related systems reported in the literature. The goal is to establish not only whether CodeSheriff detects vulnerabilities competitively, but whether its reported confidence is trustworthy, a property largely unaddressed by existing work, as discussed in Section I.

Table I reports CodeSheriff against inference baselines and external detectors on the validation split ($n=15$ claims, 7 vulnerable), at the fitted alert threshold of 0.227 and declared base rate of 3%.

CodeSheriff attains an F_1 of 0.923 at perfect precision, alerting on 6 of 15 claims. Replacing Bayesian fusion with majority voting over the identical evidence reduces F_1 to 0.142 and raises ECE from 0.027 to 0.141, since voting has no prior to moderate a single witness’s alert. The prior-only baseline reaches $ECE = 0.000$ by detecting nothing, showing ECE alone is insufficient; CodeSheriff’s Brier score of 0.013 (vs. 0.029) confirms it is calibrated *and* decisive, not merely abstaining.

Beyond these internal baselines, Table II positions CodeSheriff against three related published systems.

DeepSecure is a Python-specific hybrid deep-learning detector with a reported confidence component, and Reddy et al. is an agentic vulnerability-triage system for collaborative software repositories; both require verification of their reported metrics before their columns can be populated. MAVUL, the closest multi-agent architectural precedent, is not numerically comparable: it targets C/C++ memory-safety CWEs on a pairwise dataset using its own P-C/P-V/P-B/P-R protocol rather than precision, recall, or a calibration measure, and reports no reliability measure alongside its detection results, consistent with the gap this paper identifies in Section I.

TABLE II
COMPARISON WITH RELATED SYSTEMS. DEEPSECURE AND REDDY ET AL.
ARE PENDING VERIFICATION; MAVUL IS NOT NUMERICALLY
COMPARABLE (DIFFERENT LANGUAGE, CWE SCOPE, AND METRIC
PROTOCOL).

Parameter	DeepSecure [5]	Reddy [19]	MAVUL [18]	Ours
ECE ↓	–	–	–	0.027
Brier ↓	–	–	–	0.013
Precision ↑	–	–	–	1.000
Recall ↑	–	–	–	0.857
F_1 ↑	–	–	–	0.923

TABLE III
LEAVE-ONE-WITNESS-OUT ABLATION ON THE VALIDATION SPLIT AT
THRESHOLD 0.227. BEST PER COLUMN IN BOLD.

Configuration	ECE ↓	Brier ↓	P ↑	R ↑	F_1 ↑
Full model	0.027	0.013	1.000	0.857	0.923
– structural	0.012	0.015	1.000	0.714	0.833
– semantic	0.047	0.031	0.096	0.429	0.157
– context	0.026	0.013	1.000	0.857	0.923
– runtime	0.028	0.013	1.000	0.857	0.923

C. Ablation Study

This subsection isolates the contribution of each agent by removing it from fusion and observing the effect on the posterior. Table III removes one witness at a time; a removed witness contributes an abstention at likelihood ratio 1.0, leaving the posterior unchanged except through the evidence it no longer supplies.

The semantic witness dominates: its removal costs 0.766 in F_1 , collapses precision to 0.096, and is the only ablation that worsens calibration (ECE 0.027 $\rightarrow$ 0.047). Removing the structural witness costs 0.090 in F_1 while improving ECE, the prior-only effect in miniature: alerting less is easier to calibrate, not evidence for removal. Context and runtime ablations leave the confusion matrix unchanged on this split; both contribute measurably on the larger calibration split, so this is reported as a null result, not redundancy.

D. Assumptions and Limitations

The reported results carry several caveats that bound their generalizability. The corpus is Python-only, self-authored, and covers a closed set of ten CWEs, so reported rates describe performance on this corpus, not arbitrary pull requests. The validation split ($n=15$) is too small to attach a meaningful interval to any figure in Table I, and the held-out test split remains unevaluated at the time of writing. Evaluation is a single deterministic run against committed model recordings, so no sampling variance is reported. The structural witness ran with only one of its two backends available (Semgrep lacks a build for the fitting platform), so its likelihood ratios describe a one-backend witness. Finally, the 3% base rate is declared, not measured: the twin-paired corpus has a prevalence of 0.5 by

construction, and every weighted figure inherits and rescales with this assumption.

V. CONCLUSION

This paper presented CodeSheriff, a heterogeneous multi-agent security auditor for GitHub pull requests that fuses structural, semantic, contextual, and runtime evidence through Bayesian inference rather than majority voting. On the validation split, CodeSheriff attained an F_1 of 0.923 at perfect precision with an expected calibration error of 0.027, outperforming a majority-vote combiner over the identical evidence by a factor of 6.5 in F_1 while more than halving calibration error. The ablation study confirmed that this performance is not attributable to any single agent: the semantic witness contributes the largest share of detection power, while the structural, context, and runtime witnesses contribute complementary, non-redundant evidence. Unlike existing single-technique and multi-agent vulnerability detectors, which report a verdict with no accompanying reliability measure, CodeSheriff’s central contribution is a posterior probability that is simultaneously discriminative and calibrated, verified jointly through F_1 and Brier score rather than either alone.

FUTURE WORK

Several items remain open from Section IV-D. The structural witness must be re-observed on a Linux or CI environment where both backends, taint analysis and Semgrep, are available, since the current fit describes a one-backend witness. The Semgrep, Bandit, and single-LLM baselines require completion against the fitted artifact, and the DeepSecure and Reddy et al. comparisons require verification of their reported metrics before inclusion. The held-out test split remains unevaluated and should be scored exactly once, per the evaluation protocol in Section III-C. Beyond the current evaluation, statistical interval estimation and sensitivity analysis over the alert threshold and declared base rate would strengthen confidence in the reported figures given the small validation split. Finally, extending language coverage beyond Python and evaluating a fine-tuned semantic model against the current hosted-LLM configuration are both identified as future directions.

REFERENCES

- [1] G. Li and Y. Yang, “On the code vulnerability detection based on deep learning: A comparative study,” *IEEE Access*, 2024.
- [2] S. Han, H. Nam, J. Kang, K. Kim, S. Cho, and S. Lee, “Code-smash: Source-code vulnerability detection using siamese and multi-level neural architecture,” *IEEE Access*, 2024.
- [3] S. Han, M. Kim, J. Kang, K. Kim, S. Lee, and S. Lee, “Similarity-based source code vulnerability detection leveraging transformer architecture: Harnessing cross-attention for hierarchical analysis,” *IEEE Access*, 2024.
- [4] B. K. E. Abbasi and A. Shameli-Sendi, “Leveraging key source code metrics for predicting software vulnerabilities with machine learning,” *IEEE Access*, 2026.
- [5] R. A. Almutairi and N. Abbadeni, “Deepsecure: Hybrid deep learning framework for detecting security vulnerabilities in python code,” *IEEE Access*, 2026.

- [6] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, P. Zadorozhny, A. Cheshkov, R. Mussabayev, A. Toleu, G. Tolegen, and A. Krassovitskiy, "Finetuning large language models for vulnerability detection," *IEEE Access*, 2025.
- [7] K. Tamberg and H. Bahsi, "Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study," *IEEE Access*, 2025.
- [8] D. Gnieciak and T. Szandala, "Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection," *IEEE Access*, 2025.
- [9] B. Gülmez, "Code generation with large language models: a survey from neural program synthesis to autonomous software development," *Applied Intelligence*, vol. 56, no. 6, p. 200, 2026.
- [10] Y. Chen, Y. Shen, T. Wang, S. Ou, R. Wang, Y. Li, and Z. Pan, "Software defect detection using large language models: a literature review," *Frontiers of Computer Science*, vol. 20, no. 6, p. 2006202, 2026.
- [11] H. Su, Z. Xu, Y. Zhang, and Q. Tan, "Source code vulnerability detection based on deep learning: a review," *Cybersecurity*, vol. 9, no. 1, p. 2, 2026.
- [12] L. B. Germano, R. R. Goldschmidt, R. C. Noya, and J. C. Duarte, "A systematic review on detection, repair, and explanation of vulnerabilities in source code using large language models," *IEEE Access*, 2025.
- [13] G. Antal, L. Értekes, N. Szolnoki, and P. Hegedűs, "Evaluating retrieval-augmented generation for llm-based vulnerability detection: An empirical study on real-world java vulnerabilities," *IEEE Access*, 2026.
- [14] P.-C. Huang, C.-H. Mao, and C.-M. Lai, "Aegisguard: A context-aware framework for semantic vulnerability detection and risk stratification," *IEEE Access*, 2026.
- [15] H. Gao, D. Zhang, and L. Chen, "Scout: Llm-driven vulnerability detection via interactive repository exploration," *IEEE Access*, 2026.
- [16] I. S. Göçmen, A. S. Cezayir, and E. Tüzün, "Enhanced code reviews using pull request based change impact analysis," *Empirical Software Engineering*, vol. 30, no. 3, p. 64, 2025.
- [17] Z. Wei, J. Sun, Y. Sun, Y. Liu, D. Wu, Z. Zhang, X. Zhang, M. Li, Y. Liu, C. Li, M. Wan, J. Dong, and L. Zhu, "Advanced smart contract vulnerability detection via llm-powered multi-agent systems," *IEEE Transactions on Software Engineering*, 2025.
- [18] Y. Li, K. Joshi, X. Wang, and E. Wong, "Mavul: Multi-agent vulnerability detection via contextual reasoning and interactive refinement," in *2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2025.
- [19] K. P. Reddy, J. S. Dhatteval *et al.*, "Agentic artificial intelligence with knowledge-based reasoning for automated vulnerability triage in collaborative software repositories," in *2026 IEEE 6th International Conference on Computing, Power, and Communication Technologies (IC2PCT)*. IEEE, 2026, pp. 493–498.
- [20] T. A. Syed, M. Riyaz Belgaum, S. Jan, A. A. Khan, and S. S. Alqahtani, "Agentic ai for autonomous defense in software supply chain security: Beyond provenance to vulnerability mitigation," in *2025 International Conference on Computer and Applications (ICCA)*, 2025.
- [21] Semgrep, Inc., "Semgrep: Lightweight static analysis for many languages," <https://semgrep.dev>, accessed: Sep. 2026.
- [22] Python Code Quality Authority, "Bandit: A security linter for python source code," <https://bandit.readthedocs.io>, accessed: Sep. 2026.